



AuthiChain Production Deployment Guide

Version: 1.0.0

Date: October 2025

Status: Ready for Production



Table of Contents

1. [Pre-Deployment Checklist](#)
 2. [Environment Configuration](#)
 3. [Deployment Steps](#)
 4. [Post-Deployment Verification](#)
 5. [Monitoring & Maintenance](#)
 6. [Rollback Procedures](#)
 7. [Support & Troubleshooting](#)
-



Pre-Deployment Checklist

1. Code Quality & Testing

- ☒ All TypeScript compilation errors resolved
- ☒ Production build successful (`npm run build`)
- ☒ Critical bugs fixed:
- ☒ Subscription checkout flow working
- ☒ /product route functional
- ☒ Cannabis branding removed
- ☒ Environment variables configured
- ☒ Security review completed

2. Database

- ☐ Production database created and accessible
- ☐ Database migrations ready
- ☐ Database backup strategy in place
- ☐ Connection string secure and tested

3. Third-Party Services

- ☒ Stripe Live Mode configured
- ☒ Live API keys set
- ☒ 15 price IDs created for all tiers
- ☒ Webhooks configured
- ☒ NFT.Storage API key configured
- ☐ Email service configured (for notifications)

- [] Domain DNS configured

4. Security

- [x] All secrets in .env file
- [x] .env in .gitignore
- [x] NEXTAUTH_SECRET generated
- [x] CORS settings configured
- [] SSL/TLS certificates ready

Environment Configuration

Required Environment Variables

Create a `.env` file in your deployment with these variables:

```
# Database
DATABASE_URL="postgresql://user:password@host:5432/database?schema=public"

# Authentication
NEXTAUTH_URL="https://yourdomain.com"
NEXTAUTH_SECRET="your-super-secret-key-min-32-chars"

# Stripe (LIVE MODE - Use your production keys)
STRIPE_SECRET_KEY="sk_live_..."
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_live_..."

# Stripe Price IDs (from your Stripe Dashboard)
STRIPE_CREATOR_MONTHLY_PRICE_ID="price_..."
STRIPE_CREATOR_YEARLY_PRICE_ID="price_..."
STRIPE_PRO_MONTHLY_PRICE_ID="price_..."
STRIPE_PRO_YEARLY_PRICE_ID="price_..."
STRIPE_ENTERPRISE_MONTHLY_PRICE_ID="price_..."
STRIPE_ENTERPRISE_YEARLY_PRICE_ID="price_..."
STRIPE_AGENCY_MONTHLY_PRICE_ID="price_..."
STRIPE_AGENCY_YEARLY_PRICE_ID="price_..."

# Stripe Webhooks
STRIPE_WEBHOOK_SECRET="whsec_..."

# NFT Storage
NFT_STORAGE_API_KEY="your-nft-storage-api-key"

# Blockchain
NEXT_PUBLIC_CHAIN_ID="1"
NEXT_PUBLIC_RPC_URL="your-rpc-endpoint"

# Feature Flags
ENABLE_REFERRAL_SYSTEM="true"
ENABLE_SUBSCRIPTIONS="true"

# Site Configuration
NEXT_PUBLIC_SITE_URL="https://yourdomain.com"
NEXT_PUBLIC_SITE_NAME="AuthiChain"
```

Stripe Configuration Steps

1. **Switch to Live Mode** in Stripe Dashboard
2. **Create Products & Prices** for each tier:
 - Explorer: Free (no price needed)
 - Creator: \$29/month, \$278.4/year
 - Pro: \$79/month, \$758.4/year
 - Enterprise: \$299/month, \$2,870.4/year
 - Agency: \$999/month, \$9,590.4/year
3. **Configure Webhooks:**
 - Endpoint URL: `https://yourdomain.com/api/webhooks/stripe`
 - Events to listen for:
 - `checkout.session.completed`
 - `customer.subscription.created`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `invoice.payment_succeeded`
 - `invoice.payment_failed`



Deployment Steps

Option 1: Vercel (Recommended)

1. Connect Repository

```

```bash
Install Vercel CLI
npm install -g vercel

Login
vercel login

Deploy
cd /home/ubuntu/authichain_premium/app
vercel --prod
```

```

1. Configure Environment Variables

- Go to Vercel Dashboard → Your Project → Settings → Environment Variables
- Add all variables from your `.env` file
- Set for “Production” environment

2. Configure Domain

- Go to Settings → Domains
- Add your custom domain
- Configure DNS records as instructed

3. Deploy

```

bash
vercel --prod

```

Option 2: Railway

1. Create New Project

```
```bash
Install Railway CLI
npm install -g @railway/cli

Login
railway login

Initialize
cd /home/ubuntu/authichain_premium/app
railway init
```
```

1. Add Environment Variables

```
```bash
Add variables via CLI
railway variables set KEY=VALUE

Or use Railway Dashboard
```
```

1. Deploy

```
bash
railway up
```

Option 3: Custom Server (VPS/AWS/GCP)

1. Server Requirements

- Node.js 18+ installed
- PostgreSQL database accessible
- Nginx as reverse proxy
- SSL certificate (Let's Encrypt)

2. Build Application

```
bash
cd /home/ubuntu/authichain_premium/app
npm install
npm run build
```

3. Configure Process Manager (PM2)

```
```bash
Install PM2
npm install -g pm2

Start application
pm2 start npm --name "authichain" -- start

Save configuration
pm2 save
pm2 startup
```
```

1. Configure Nginx

```
```nginx
```

```
server {
 listen 80;
 server_name yourdomain.com;
 return 301 https://$server_name$request_uri;
}
```

```
server {
 listen 443 ssl http2;
 server_name yourdomain.com;
```

```
 ssl_certificate /path/to/cert.pem;
 ssl_certificate_key /path/to/key.pem;

 location / {
 proxy_pass http://localhost:3000;
 proxy http version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Connection 'upgrade';
 proxy_set_header Host $host;
 proxy_cache_bypass $http_upgrade;
 }
```

```
}
...
```

## 🌟 Post-Deployment Verification

### 1. Functionality Checks

**Test these critical flows:**

```
Homepage loads
curl -I https://yourdomain.com

Pricing page loads
curl -I https://yourdomain.com/pricing

API health check
curl https://yourdomain.com/api/nft/list

Marketplace loads
curl -I https://yourdomain.com/marketplace
```

#### Manual Testing:

- [ ] Homepage loads correctly
- [ ] User signup flow works
- [ ] Wallet authentication works
- [ ] Pricing page displays correctly
- [ ] “Get Started” buttons trigger Stripe checkout
- [ ] Marketplace shows products (not cannabis categories)
- [ ] Dashboard loads after authentication

- [ ] NFT minting works
- [ ] /product route redirects to /explore

## 2. Stripe Integration Test

### 1. Test Checkout Flow:

- Click “Get Started” on a paid tier
- Complete test payment with Stripe test card: 4242 4242 4242 4242
- Verify redirect to dashboard with success message

### 2. Verify Webhook:

- Check Stripe Dashboard → Webhooks
- Confirm events are being received
- Check application logs for webhook processing

## 3. Database Verification

```
Check database connectivity
psql $DATABASE_URL -c "SELECT COUNT(*) FROM \"User\";"

Verify tables exist
psql $DATABASE_URL -c "\dt"
```

## 4. Performance Check

```
Test page load times
curl -w "@curl-format.txt" -o /dev/null -s https://yourdomain.com

Where curl-format.txt contains:
time_namelookup: %{time_namelookup}\n
time_connect: %{time_connect}\n
time_appconnect: %{time_appconnect}\n
time_pretransfer: %{time_pretransfer}\n
time_redirect: %{time_redirect}\n
time_starttransfer: %{time_starttransfer}\n
-----\n
time_total: %{time_total}\n
```



## Monitoring & Maintenance

### Application Monitoring

#### 1. Error Tracking

- Set up Sentry or similar service
- Monitor error rates
- Set up alerts for critical errors

#### 2. Performance Monitoring

- Use Vercel Analytics or Google Analytics
- Monitor page load times
- Track Core Web Vitals

### 3. Uptime Monitoring

- Set up UptimeRobot or Pingdom
- Monitor critical endpoints
- Configure SMS/Email alerts

## Database Monitoring

```
Check database size
psql $DATABASE_URL -c "SELECT pg_size_pretty(pg_database_size('your_db'));"

Monitor active connections
psql $DATABASE_URL -c "SELECT count(*) FROM pg_stat_activity;"

Check slow queries
psql $DATABASE_URL -c "SELECT query, mean_exec_time FROM pg_stat_statements ORDER BY
mean_exec_time DESC LIMIT 10;"
```

## Log Management

```
View application logs (PM2)
pm2 logs authichain

View Nginx access logs
tail -f /var/log/nginx/access.log

View Nginx error logs
tail -f /var/log/nginx/error.log
```

---

## Rollback Procedures

### Vercel Rollback

```
List deployments
vercel ls

Rollback to previous deployment
vercel rollback [deployment-url]
```

### Railway Rollback

1. Go to Railway Dashboard
2. Select your project
3. Click "Deployments"
4. Find previous working deployment
5. Click "Redeploy"

## Custom Server Rollback

```
If using Git deployment
cd /home/ubuntu/authichain_premium/app
git log --oneline -10
git checkout [previous-commit-hash]
npm install
npm run build
pm2 restart authichain
```

## Support & Troubleshooting

### Common Issues

#### 1. Build Fails with TypeScript Errors

```
Solution: Clear Next.js cache
rm -rf app/.next
npm run build
```

#### 2. Database Connection Timeout

```
Check if DATABASE_URL is correct
echo $DATABASE_URL

Test connection
psql $DATABASE_URL -c "SELECT 1;"
```

#### 3. Stripe Webhooks Not Working

- Verify webhook URL is publicly accessible
- Check webhook secret matches environment variable
- Review Stripe Dashboard → Webhooks → Event logs

#### 4. NFT Minting Fails

- Verify NFT\_STORAGE\_API\_KEY is valid
- Check NFT.Storage API status
- Review application logs for errors

### Getting Help

- **Documentation:** See other guides in this directory
- **Stripe Support:** <https://support.stripe.com>
- **Next.js Docs:** <https://nextjs.org/docs>
- **Prisma Docs:** <https://www.prisma.io/docs>



## Final Notes

### Post-Launch Tasks

- [ ] Set up automated database backups



- [ ] Configure CDN for static assets
- [ ] Set up monitoring and alerting
- [ ] Create runbook for common operations
- [ ] Document any custom configurations
- [ ] Set up staging environment for testing updates

## Security Best Practices

- Regularly update dependencies
- Monitor security advisories
- Rotate secrets periodically
- Review access logs regularly
- Keep backups secure and tested

---

**Deployment Date:** \_\_\_\_

**Deployed By:** \_\_\_\_

**Production URL:** \_

**Notes:** \_\_\_\_

---



## Congratulations!

---

Your AuthiChain NFT marketplace is now live in production! 🚀

For ongoing support and updates, refer to the complete documentation suite in this directory.